

CS 61A 暑期 2026

第 01 讲：编码环境、 函数与异常（一）

课程讲义 · 中文版（精简版）

CS 61A: 计算机程序的结构与解释

- 你将学习 **3 门编程语言**：Python、Scheme 和 SQL
- ……但这**不只是一门编程入门课**
- 核心理念：**抽象、问题求解、管理复杂性、解释编程语言**
- ……换句话说，学习**像计算机科学家一样思考**

如何取得好成绩？

- 全程参与所有同步环节（讲座、实验、讨论课）
- 独立完成作业，卡住时主动求助——经历**有成效的挣扎**
- 考试：做历年真题

AI 政策速览（TLDR）：

用 AI 替你写代码 **✗ 禁止**

用 AI 当文档/学习工具 **✓ 允许**

为什么？

- 我们知道 AI 编程工具在业界被广泛使用——但你想成为那个整天无脑"vibe coding"的人吗？
- 你想把自己的思考和技能依赖在一家公司身上吗（它更关心利润而不是你）？
- 如果你连基础知识都没有，怎么判断 AI 输出的代码是**好的、正确的**？
- 批判性思维、问题求解、设计、沟通等能力依然重要。
- 归根结底，**作为工程师，你要对所有 AI 输出负责。**

第一部分

编码环境

文件系统 · 终端 · Python 解释器

文件 (File)： 存储数据的基本单元

目录 (Directory / 文件夹)： 组织文件的容器

路径 (Path)： 文件或目录的位置

绝对路径

从根目录开始的完整地址

例如：

`/home/user/projects/lab01.py`

类比：你的完整通信地址

相对路径

相对于当前所在目录

例如：

`./lab01.py`

类比："我住在往北三个街区的公寓"

终端 (Terminal) 常用命令

命令行 (文本) 界面，用于与计算机和文件系统交互

`pwd` # 显示当前工作目录 (print working directory)

`ls` # 列出文件和目录 (list)

`cd` <目录> # 切换目录 (change directory)

`cd ..` # 进入上级目录

`cd ~` # 回到主目录 (home)

`touch` <文件> # 创建空文件

`mkdir` <目录> # 创建目录 (make directory)

`rm` <文件> # 删除文件或目录 (危险! 不可逆)

↑ / ↓ 方向键: 翻阅历史命令

安装 Python 3 后，在终端输入 `python3` 即可启动交互模式

终端 vs. 解释器

- **终端**：执行系统命令（文件操作、目录管理等）
- **解释器**：执行 Python 代码，逐行即时返回结果

退出解释器：`exit()`、`quit()`，或按 **Ctrl + D**

解释器 vs. IDE（如 VS Code）

- **解释器**：临时运行、快速验证小段代码
- **IDE（集成开发环境）**：在文件中编写较长代码，适合软件开发，有更多辅助功能（补全、调试等）

第二部分

函数

基本数据类型 · 表达式 · 名字与赋值

函数定义 · 调用求值 · print 与 return

基本数据类型

整数 (Integer) : `-5`, `0`, `100`

浮点数 (Float) : `-1.1`, `0.0`, `43.9235`

字符串 (String) : `'hello world'` 或 `"hello world"`

三引号 (`'''` 或 `"""`) 用于多行字符串或注释

布尔值 (Boolean) : `True` 或 `False`

表达式 (Expressions)

将数据类型组合起来的方式。Python 会自动计算（求值）表达式。`#` 表示单行注释。

```
>>> (2 + 2) * 5
```

```
20
```

```
>>> 5 ** 2 # 幂运算
```

```
25
```

```
>>> 10 / 3 # 真除法
```

```
3.3333333333333335
```

```
>>> 10 // 3 # 整除
```

```
3
```

```
>>> 10 % 3 # 取模
```

```
1
```

```
>>> 'hello'
```

```
'hello'
```

```
>>> 'hello' + 'world'
```

```
'helloworld'
```

```
>>> 'a' * 5
```

```
'aaaaa'
```

名字与赋值 (Names and Assignment)

给值起个标签方便以后使用，称为将值绑定到**名字**（常叫**变量**）。

```
>>> pi = 3.1415926
>>> pi
3.1415926
```

```
>>> # 多重赋值
>>> x, y = 1, 3
>>> x
1
>>> y
3
```

```
>>> name = 'Rebecca'
>>> f'Hello {name}!' # f-string
'Hello Rebecca!'
```

```
>>> # 导入语句
>>> from operator import mul
>>> mul(10, 5)
50
```

定义函数 (1/2)

- 想把一组操作抽象出来反复使用？→ 自己定义**函数**！
- 函数可以接收 0 个或多个输入（**参数 parameters**），返回 1 个或多个输出（**返回值 return values**）。
- 调用时传入的具体值称为**实参 (argument)**（例如下面的 5）。
- **调用（使用）函数**构成一个调用表达式：函数名 + 括号内逗号分隔的实参。

```
>>> from math import pi
>>> def circle_area(radius):
...     return pi * (radius ** 2)
>>> circle_area(5)
78.53981633974483
```

定义函数 (2/2)

- 函数可以返回多个值（逗号分隔）
技术上是单个元组 tuple，以后会讲
- 参数可以有默认值
- 类型提示 (Type hints)：告诉解释器和程序员每个参数和返回值的数据类型
- 文档字符串 (Docstrings)：函数签名下方的多行字符串，说明函数行为
可以包含 doctest——展示函数预期行为的示例代码

```
def circle_area(radius: float = 1) -> float:
    """Returns the area of a circle
    with the given radius.

    >>> from math import pi
    >>> circle_area()
    3.141592653589793
    >>> circle_area(5)
    78.53981633974483
    """
    return pi * (radius ** 2)
```

内置函数 (Builtins)

Python 自带若干内置函数，无需 import 即可直接使用：

```
>>> max(1, 2, 3)      # 最大值
3
>>> min(1, 2, 3)     # 最小值
1
>>> len('hello')      # 长度
5
>>> sum(1, 2, 3)      # 求和
6
>>> print('hello world') # 打印输出
hello world
```

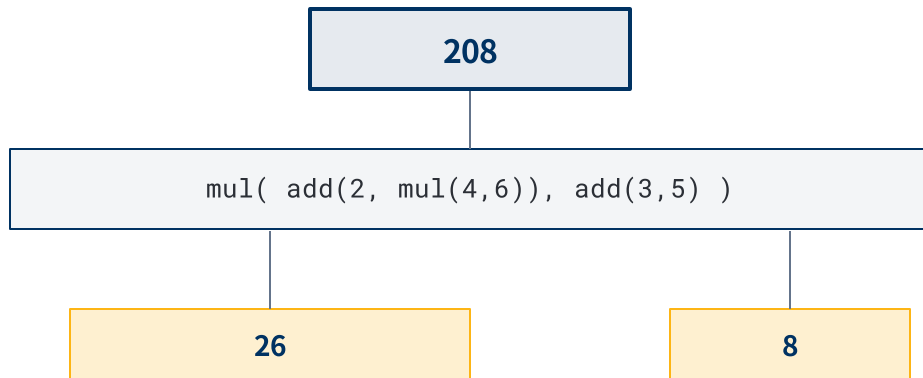
三步求值规则：

1. 求运算符 (**operator**)：函数名是否存在？
2. 从左到右求实参 (**operands**)：把每个参数化简为单个值
 - a. 这里可能需要递归执行步骤 1-3！ (见下一页)
3. 将运算符作用于实参：运行函数体中的代码

```
max ( 7.5 , 9.5 )  
operator operand operand
```

调用表达式求值示例（2/2）

`mul(add(2, mul(4, 6)), add(3, 5))`



求值过程（从最内层开始）：

① `mul(4, 6) → 24`

② `add(2, 24) → 26` ③ `add(3, 5) → 8`

④ `mul(26, 8) → 208`

注意：先算最内层的嵌套调用，再逐层向外。

print vs. return

- 想用屏幕上**显示**东西 → 用 `print`
- 想真正**计算出**一个值并供后续使用 → 用 `return`

类比：print 就像我有 5 块钱，掏出来给你看；return 就像我把 5 块钱交到你手里让你花。

纯函数 vs. 非纯函数

- **纯函数：**只返回值。给定相同输入，永远产生相同输出，无副作用。
- **非纯函数：**产生副作用，比如修改（mutate）某个数据结构，或打印输出。

None

- 如果函数没有 `return` 语句会怎样?
 - 函数会隐式返回 `None`——Python 中代表"空无"的特殊值
- `print` 函数的返回值也是 `None`

注意：None 不会被解释器自动显示，除非显式 print 出来！

```
>>> def square(x):  
...     x ** 2          # 没有 return!  
>>> square(2)          # 什么都不显示  
>>> print(square(2))  
None
```

第三部分

异常（一）

异常类型 · 阅读堆栈跟踪
Print 调试 · assert 语句

异常类型

程序运行出错时会产生**异常（exception / error）**，也称"抛出"（raised / thrown）。

按调试难度从易到难分为三类：

语法错误

Syntax

"非法的" Python 代码。
解释器根本无法解析运行。

最容易发现和修复

运行时错误

Runtime

代码语法合法，但运行时做了导致程序崩溃的操作。

中等难度

逻辑错误

Logical

代码合法、不崩溃，但输出不是你想要的结果。

最难调试——没有报错提示

阅读堆栈跟踪 (Stack Trace)

运行时错误会打印一段堆栈跟踪，**从下往上读**（最早的调用在最下面）：

```
Traceback (most recent call last):
```

```
File "/path/to/example.py", line 4, in <module>
```

```
    greet('Chad')
```

```
... ← 最上层：你调用的地方
```

```
File "/path/to/example.py", line 2, in greet
```

```
    print('Hello, ' + someon)
```

↑ 错误真正发生的行

NameError: name 'someon' is not defined

↑ 最底部：错误类型和原因（someon 拼写错误，应为 someone）

Print 调试

- 最基础的调试技巧：在关键位置插入 `print` 语句，打印你关心的变量值，观察程序实际走到了哪一步。
- 作业中建议在调试 `print` 前加 `'DEBUG: '` 前缀，这样 OkPy 自动评分器不会误判。

```
>>> from math import sqrt
>>> def quadratic_formula_positive(a, b, c):
...     discriminant = b * 2 - 4 * a * c
...     print("DEBUG: discriminant is", discriminant)
...     temp = (-b + sqrt(discriminant)) / (2 * a)
...     print("DEBUG: temp is", temp)
...     return temp
```

Assert 语句（防御性编程）

与其等 bug 出现再调试，不如**预防**——这就是**防御性编程**。

用 `assert` 语句在代码里程序化地验证你的假设。如果假设为 `False`，会抛出 `AssertionError`（可附带你写的错误信息）。

```
>>> def divide(a, b):
...     assert b != 0, 'divisor cannot be zero'
...     return a / b
>>> divide(5, 0)
Traceback (most recent call last):
  File "<python-input-1>", line 1, in <module>
    divide(5, 0)
  File "<python-input-0>", line 2, in divide
    assert b != 0, 'divisor cannot be zero'
           ^
AssertionError: divisor cannot be zero
```